SAECOIN

TESTING STRATEGY

Version 1.0

Project: SAECOIN
Ticker: SAC (Provisional)
Document: Testing Strategy
Status: Planning
Blockchain Type: Independent Layer-1
Consensus: Proof of Work (PoW)
Transaction Model: UTXO
Development Environment: Windows 11, 16 GB RAM

---

1. PURPOSE

This document defines the complete testing strategy for SAECOIN.

The objective is to ensure that the blockchain, node software, wallets, mining software, APIs, networking, and explorer work correctly and securely.

Testing must identify problems before they can affect users or cause financial loss.

No component should be considered production-ready merely because it compiles or runs successfully.

2. TESTING PRINCIPLES

All testing must follow these principles:

1. Correctness before performance.
2. Security before convenience.
3. Consensus consistency across all nodes.
4. Reproducible test results.
5. No undocumented consensus changes.
6. Automated regression testing.
7. Testnet validation before mainnet.
8. No real funds on development or test networks.
9. Private keys and seed phrases must never appear in test logs.
10. Every critical bug must be resolved and retested before release.

3. TESTING ENVIRONMENTS

3.1 Local Development

Purpose:

- Test individual modules.
- Run unit and integration tests.
- Debug blockchain operations.
- Test wallets and mining software.
- Verify API responses.

Environment:

- Windows 11.
- 16 GB RAM.
- Rust development environment.
- Android development environment.
- Local blockchain database.
- Local development network.

3.2 Private Testnet

Purpose:

- Connect multiple independent nodes.
- Test P2P communication.
- Test block propagation.
- Test mining.
- Test transaction synchronization.
- Test network failures.
- Test chain reorganizations.

Testnet must use separate network identifiers, genesis block, addresses, and configuration from mainnet.

3.3 Mainnet Pre-Launch Testing

Purpose:

- Final consensus verification.
- Security testing.
- Recovery testing.
- Performance benchmarking.
- Independent audit.
- Release validation.

Mainnet launch must remain blocked until all critical release requirements are satisfied.

4. TESTING LEVELS

4.1 Unit Testing

Test individual functions independently.

Examples:

- Hash calculation.
- Transaction serialization.
- Address validation.
- Signature verification.
- Block reward calculation.
- Difficulty calculation.
- UTXO lookup.
- Fee calculation.

4.2 Integration Testing

Test interactions between modules.

Examples:

- Wallet to node.
- Miner to node.
- Node to node.
- Transaction to mempool.
- Block to database.
- Explorer to blockchain index.

4.3 End-to-End Testing

Test complete user workflows.

Examples:

1. Create wallet.
2. Generate address.
3. Receive test SAC.
4. Wait for confirmations.
5. Send SAC.
6. Verify transaction.
7. Check updated balance.

4.4 Regression Testing

Every bug fix must include a regression test.

Previously working functionality must remain operational after new updates.

5. CONSENSUS TESTING

Consensus is one of the most critical parts of SAECOIN.

All compatible full nodes must independently reach the same valid chain.

Tests must include:

5.1 Serialization

- Deterministic block serialization.
- Deterministic transaction serialization.
- Identical data must produce identical bytes.
- Different data must not accidentally produce identical encodings.
- Invalid or incomplete serialization must be rejected.

5.2 Hashing

- Verify block hash calculations.
- Verify transaction identifiers.
- Verify header hashing.
- Verify mining hash calculations.
- Cross-check results between independent implementations where possible.

The final cryptographic algorithms must be selected and documented before consensus implementation is finalized.

5.3 Proof of Work

Test:

- Valid proof acceptance.
- Invalid proof rejection.
- Target comparison.
- Nonce handling.
- Block header validation.
- Mining difficulty changes.
- CPU/GPU solution verification.

The full node must independently verify every submitted block.

5.4 Block Validation

Test:

- Valid blocks.
- Invalid blocks.
- Incorrect previous block hash.
- Incorrect transaction root.

- Invalid timestamp.
- Invalid block reward.
- Invalid transaction.
- Invalid proof of work.
- Duplicate transactions.
- Invalid coinbase transaction.

5.5 Difficulty Adjustment

Test:

- Normal block intervals.
- Extremely fast block production.
- Extremely slow block production.
- Sudden hashrate changes.
- Timestamp manipulation.
- Difficulty boundary conditions.

The final difficulty adjustment formula must be specified before these tests become protocol acceptance tests.

5.6 Block Reward and Supply

Proposed parameters:

- Initial reward: 50 SAC.
- Halving interval: 210,000 blocks.
- Proposed maximum supply: 21,000,000 SAC.
- Decimal precision: 8.

Tests must verify:

- Correct initial reward.
- Correct halving boundaries.
- Correct integer rounding.
- Correct coinbase reward.
- Correct fee inclusion.
- No unauthorized coin creation.
- No integer overflow.
- No supply exceeding the maximum.

Important: The proposed reward and halving schedule must undergo mathematical verification. The implementation must calculate theoretical total issuance using the smallest unit and prove that the actual supply does not exceed the intended cap. Do not assume the proposed parameters automatically produce exactly 21 million SAC.

5.7 Chain Selection

Test:

- Valid chain selection.
- Cumulative work comparison.
- Competing blocks.
- Competing chains.
- Chain reorganization.
- Recovery after restart.
- Invalid chain rejection.

The final chain selection rules must be identical across all full nodes.

6. UTXO AND TRANSACTION TESTING

Test transaction processing thoroughly.

Required cases:

- Valid transaction.
- Invalid signature.
- Missing input.
- Invalid output.
- Insufficient input value.
- Input/output value mismatch.
- Negative amount.
- Zero-value edge cases.
- Double spending.
- Duplicate inputs.
- Invalid transaction format.
- Excessive transaction size.
- Fee calculation.
- Integer overflow.
- Dust handling, if defined by the protocol.
- Coinbase maturity.
- Mempool conflicts.

The node must never accept a transaction that creates unauthorized value.

7. P2P NETWORK TESTING

Test communication between independent nodes.

7.1 Basic Networking

- Peer discovery.
- Peer connection.
- Peer disconnection.
- Message exchange.

- Block propagation.
- Transaction propagation.
- Peer synchronization.

7.2 Failure Testing

- Network interruption.
- Slow connections.
- Connection timeout.
- Node restart.
- Peer restart.
- Network partition.
- Reconnection.
- Synchronization after downtime.

7.3 Malicious Peer Testing

- Malformed messages.
- Oversized messages.
- Invalid blocks.
- Invalid transactions.
- Repeated duplicate messages.
- Excessive connection attempts.
- Message flooding.
- Invalid protocol versions.

Nodes must safely reject invalid data without crashing.

8. DATABASE TESTING

The blockchain database must preserve data integrity.

Test:

- Block storage.
- Block retrieval.
- Transaction indexing.
- UTXO updates.
- Atomic database writes.
- Database restart.
- Unexpected application shutdown.
- Crash recovery.
- Database corruption detection.
- Reindexing.
- Undo operations.
- Redo operations.
- Chain reorganization recovery.

No partially committed blockchain state should become visible after a failed database operation.

9. WALLET TESTING

Wallet testing must cover Android and Windows applications.

9.1 Wallet Creation

- Generate a new wallet.
- Generate a valid address.
- Validate address format.
- Verify key generation.
- Verify backup creation.

9.2 Security

- Private keys must remain private.
- Seed phrases must not appear in logs.
- Encrypted storage must work correctly.
- Password protection must be tested.
- Failed unlock attempts must be handled safely.
- Backup and restore must preserve wallet access.
- Local transaction signing must work without exposing private keys to remote APIs.

9.3 Transaction Operations

- Receive SAC.
- Display balance.
- Display transaction history.
- Send SAC.
- Calculate fees.
- Handle insufficient funds.
- Handle invalid addresses.
- Handle network mismatch.
- Display confirmation status.

9.4 Platform Testing

Android:

- Supported Android versions.
- App restart.
- Background and foreground behavior.
- Network interruptions.
- Local storage security.
- Wallet backup and restore.

Windows:

- Installation.
- Startup.
- Wallet creation.
- Wallet backup.
- Transaction signing.
- Application restart.
- Database access.
- Software update behavior.

10. MINING TESTING

Mining must be tested independently and against the full node.

10.1 CPU Mining

Test:

- Mining initialization.
- CPU thread configuration.
- Nonce generation.
- Hashrate calculation.
- Valid solution discovery.
- Invalid solution rejection.
- Miner shutdown.
- Miner restart.

10.2 GPU Mining

Test:

- GPU detection.
- Device initialization.
- Mining correctness.
- Memory handling.
- GPU temperature and utilization reporting where available.
- Device failure.
- Driver failure.
- Miner restart.

10.3 CPU/GPU Consistency

CPU and GPU miners must produce solutions that the full node can verify.

The full node must use the same consensus verification rules regardless of the mining hardware.

10.4 Mining Modes

Test:

- Solo mining.
- Pool mining integration, if implemented.
- Mining reward calculation.
- Stale solutions.
- Invalid shares.
- Block submission.
- Block acceptance and rejection.

Pool shares must not be confused with valid blockchain blocks.

11. API TESTING

Test every documented API endpoint.

Required checks:

- Correct HTTP methods.
- Correct response formats.
- Correct error responses.
- Input validation.
- Output validation.
- API versioning.
- Rate limiting.
- Authentication.
- Authorization.
- Request size limits.
- Network identification.
- Invalid transaction broadcast.
- Duplicate transaction broadcast.
- Node synchronization status.

APIs must not expose private keys or wallet seed phrases.

12. BLOCK EXPLORER TESTING

Verify that explorer information matches the active blockchain.

Test:

- Block height.
- Block hash.
- Transaction details.
- Transaction status.
- Address history.

- Confirmations.
- Current supply.
- Network hashrate estimates.
- Reorganization handling.
- Index rebuilding.

The explorer must recover its indexes from the authoritative blockchain data if necessary.

13. SECURITY TESTING

Security testing is mandatory before mainnet.

13.1 Static Analysis

Review source code for:

- Unsafe operations.
- Memory safety issues.
- Integer overflow.
- Improper error handling.
- Insecure randomness.
- Sensitive data exposure.

13.2 Dependency Auditing

- Audit Rust dependencies.
- Review Android dependencies.
- Review desktop dependencies.
- Check known vulnerabilities.
- Remove unused dependencies.
- Avoid untrusted packages.

13.3 Fuzz Testing

Fuzz:

- Transaction parsers.
- Block parsers.
- Network message parsers.
- Address parsers.
- API input handlers.
- Serialization and deserialization.

Unexpected input must not cause crashes, memory corruption, or unauthorized state changes.

13.4 Secret Scanning

Check repositories for:

- Private keys.
- Seed phrases.
- API secrets.
- Passwords.
- Database credentials.
- Signing keys.

13.5 Independent Security Audit

Before mainnet:

- Review consensus implementation.
- Review wallet security.
- Review cryptographic implementation.
- Review P2P attack resistance.
- Review database recovery.
- Review supply enforcement.

Critical findings must be resolved before launch.

14. PERFORMANCE TESTING

Performance must be measured rather than assumed.

Measure:

- Block validation time.
- Transaction validation time.
- Block propagation time.
- Synchronization speed.
- Database read/write performance.
- Memory usage.
- CPU utilization.
- GPU utilization.
- Wallet response time.
- Explorer indexing speed.

The initial development environment is Windows 11 with 16 GB RAM.

Tests should record actual resource consumption and identify practical limitations.

Do not establish unrealistic performance targets before benchmarking the first working implementation.

15. COMPATIBILITY TESTING

Verify that different implementations produce identical consensus results.

Test combinations of:

- Different full nodes.
- Different operating systems.
- Different CPU architectures where supported.
- CPU miners.
- GPU miners.
- Android wallets.
- Windows wallets.

All compatible nodes must agree on block validity and chain selection.

16. TEST AUTOMATION AND CI

Use automated tests wherever practical.

The GitHub repository should eventually include an automated CI workflow.

Suggested checks:

1. Rust formatting.
2. Rust compilation.
3. Unit tests.
4. Integration tests.
5. Static analysis.
6. Dependency audit.
7. Android build.
8. Desktop build.
9. Regression tests.

A failed critical test must prevent the relevant release build from being approved.

17. TEST CASE DOCUMENTATION

Each important test should record:

- Test ID.
- Test name.
- Module.
- Preconditions.
- Input data.
- Expected result.
- Actual result.
- Pass or fail.
- Software version.
- Test environment.

- Date.
- Related bug ID.

Example:

Test ID: CONS-001
Test Name: Valid Block Verification
Module: Consensus
Input: A correctly formed block with valid proof of work.
Expected Result: Full node accepts the block.
Actual Result: To be recorded.
Status: Pending.

18. RELEASE ACCEPTANCE CRITERIA

18.1 Development Release

Requirements:

- Project builds successfully.
- Unit tests pass.
- Core modules operate.
- Local node starts.
- Basic wallet functionality works.

18.2 Testnet Release

Requirements:

- Multiple independent nodes connect.
- Blocks propagate correctly.
- Transactions synchronize.
- Mining works.
- Chain reorganizations work correctly.
- Node restart and recovery work.
- No unresolved critical consensus defects.

18.3 Mainnet Release

Requirements:

- Consensus specification finalized.
- Genesis block finalized.
- Supply mathematics independently verified.
- Consensus test vectors published.
- All critical consensus tests pass.
- Security testing completed.
- Independent security audit completed.

- Wallet backup and recovery tested.
- Database recovery tested.
- Network attack resistance reviewed.
- Mainnet and testnet separation verified.
- Release binaries reproducibly built where practical.
- No unresolved critical or high-severity security issues.
- Mainnet launch checklist approved.

19. BUG SEVERITY CLASSIFICATION

Critical

- Unauthorized coin creation.
- Consensus failure.
- Private key exposure.
- Invalid blocks accepted.
- Funds can be stolen or irrecoverably lost.

High

- Major transaction failure.
- Serious network vulnerability.
- Wallet backup or recovery failure.
- Major database corruption.

Medium

- Partial functionality failure.
- Performance degradation.
- Non-critical synchronization problems.

Low

- Minor UI issues.
- Cosmetic problems.
- Non-critical display errors.

Critical issues must be resolved before any production release.

20. TESTNET RULES

- Testnet coins have no guaranteed monetary value.
- Never use mainnet private keys for testing.
- Never mix mainnet and testnet data.
- Clearly label testnet applications.
- Testnet may be reset during development.
- Testnet results must be documented.
- Mainnet launch must not depend only on successful local tests.

21. FINAL TESTING POLICY

SAECOIN must follow this development sequence:

1. Build individual modules.
2. Run unit tests.
3. Run integration tests.
4. Run local end-to-end tests.
5. Run automated regression tests.
6. Deploy private testnet.
7. Conduct network and mining tests.
8. Conduct security and recovery tests.
9. Complete independent audits.
10. Approve mainnet release.

A successful software build does not prove that a blockchain is secure.

SAECOIN must not launch its mainnet until consensus correctness, supply enforcement, network reliability, wallet security, and recovery procedures have been independently validated.

END OF DOCUMENT